

Cloud, DevOps & Cybersecurity — Professional Masterclass

Deploy resilient systems and secure them from infrastructure to application.

This is an original, project-first professional course manual created for educational and commercial course-development use.

The material is designed to be taught, adapted, branded and expanded. It emphasizes practical engineering, portfolio projects, assessments and production thinking rather than certificate collection. Technology names are used for educational context; specific provider pricing, product limits and interfaces should be verified before publication because they change over time.

Course Architecture

- 7 large modules with lessons, practical exercises, assessments and projects.
- Recommended delivery: 10–14 weeks at 6–10 hours per week, or self-paced.
- Primary outcome: a portfolio-ready capstone plus reusable professional skills.
- Suggested teaching method: concept → demonstration → guided lab → independent project → assessment.

Module Roadmap

1	Linux, Networking and Systems
2	Cloud Architecture
3	Docker, Kubernetes and CI/CD
4	Application and Cloud Security
5	Monitoring, SRE and Incident Response
6	Advanced Infrastructure and Zero-Trust Thinking
7	Capstone: Secure Production Platform

Module 1 — Linux, Networking and Systems

Learning objectives

- Linux command line
- Processes and services
- Files and permissions
- TCP/IP basics
- DNS and HTTP
- Ports and firewalls
- System troubleshooting

Core lesson

Cloud and security work begins with systems fundamentals. Learn how Linux processes run, how files are permissioned, how services start and how logs reveal failures.

Networking concepts include IP addresses, subnets, routing, DNS, TCP, TLS and HTTP. You do not need to become a network engineer, but you should understand what happens when a browser connects to a server.

Use command-line tools to inspect processes, ports, disk space, memory and network connectivity. Troubleshooting should be hypothesis-driven: observe, isolate, test and verify.

Permissions should follow least privilege. Give users and services only the access they require. Avoid running application processes with unnecessary administrative privileges.

Security is easier when the underlying system is understood. Many application failures that look like code bugs are actually configuration, permissions, networking or resource problems.

Practical lab

- Create a Linux VM or safe lab environment.
- Practice process and network inspection.
- Configure a least-privilege service account.
- Trace an HTTP request using command-line tools.
- Write a troubleshooting checklist.

Assessment

Assessment: Explain DNS, TCP and HTTPS in the path from browser to server.

Module project

Project: Linux Operations Lab — deploy a small service, configure permissions, inspect logs and diagnose controlled failures.

Module 2 — Cloud Architecture

Learning objectives

- Compute
- Storage
- Managed databases
- Virtual networking
- Identity and access management
- Load balancing
- High availability

Core lesson

Cloud platforms expose infrastructure through APIs. The names differ across providers, but the architectural concepts are similar: compute, storage, networking, identity and managed services.

Compute can be virtual machines, containers, serverless functions or managed application platforms. Choose based on operational needs rather than fashion.

Object storage is suited to files and large unstructured data. Databases provide structured persistence. Backups and retention must be designed separately from the application.

Virtual networks isolate resources and control traffic. Public exposure should be minimized. Place databases and internal services behind appropriate network boundaries.

Identity and access management is a major security layer. Use roles and short-lived credentials where possible. Avoid sharing administrator credentials among developers.

High availability requires redundancy and a plan for failure. A single instance is a single point of failure. However, redundancy has cost, so design according to actual availability requirements.

Practical lab

- Design a three-tier cloud architecture.
- Create least-privilege roles.
- Separate public and private resources.
- Configure object storage with controlled access.
- Document backup and recovery requirements.

Assessment

Assessment: Design a cloud architecture for a small SaaS and identify all single points of failure.

Module project

Project: Cloud Reference Architecture — compute, storage, database, networking, IAM and backup design.

Module 3 — Docker, Kubernetes and CI/CD

Learning objectives

- Container images
- Dockerfiles
- Registries
- Kubernetes concepts
- Deployments and services
- CI/CD pipelines
- Infrastructure as code

Core lesson

Containers package applications consistently across environments. A Dockerfile should be deterministic, avoid secrets and use a sensible base image. Multi-stage builds can reduce production image size.

Kubernetes orchestrates containers at scale. Learn the basic concepts: pods, deployments, services, configuration, secrets, health probes and namespaces. Do not introduce Kubernetes simply because a project is small; it adds operational complexity.

CI/CD should automatically test and build changes. A deployment pipeline can include code quality checks, security scanning, unit tests, integration tests and staged deployment.

Infrastructure as code represents infrastructure declaratively. It makes environments reproducible and reviewable. Keep state secure and protect production changes.

The objective of DevOps is not “more tools.” It is shorter, safer feedback loops between development and operations.

Practical lab

- Containerize an application.
- Build a CI pipeline.
- Deploy a service to a local Kubernetes cluster.
- Add health checks and resource limits.
- Represent a small environment as infrastructure code.

Assessment

Assessment: Compare VM, container and Kubernetes deployment choices for three different workloads.

Module project

Project: CI/CD Platform — source control, automated tests, container build, deployment and rollback procedure.

Module 4 — Application and Cloud Security

Learning objectives

- Threat modeling
- IAM
- Secrets management
- Network security
- Web application risks
- Dependency security
- Security logging

Core lesson

Security should be designed as layers. Start with threat modeling: identify assets, actors, trust boundaries, attack paths and impacts. Then choose controls.

Identity and access management should enforce least privilege. Separate human administration from application identities. Rotate credentials and use managed secret stores where available.

Network security reduces unnecessary exposure. Limit inbound traffic, segment sensitive services and protect administrative interfaces.

Web applications commonly face injection, broken authorization, insecure file handling, credential theft and dependency vulnerabilities. Validate inputs, encode outputs where required and test authorization directly.

Dependency security matters because applications inherit risks from libraries and container images. Pin versions where appropriate, scan dependencies and patch important vulnerabilities promptly.

Logs should help detect suspicious activity without becoming a privacy liability. Protect log access and avoid recording secrets.

Practical lab

- Threat-model a deployed application.
- Create least-privilege IAM roles.
- Move secrets into a secret-management system.
- Run dependency and container scans.
- Define security events that should trigger alerts.

Assessment

Assessment: Produce a threat model and layered defense plan for a public SaaS application.

Module project

Project: Security Hardening Sprint — identify and remediate at least ten security weaknesses in a safe lab application.

Module 5 — Monitoring, SRE and Incident Response

Learning objectives

- Metrics, logs and traces
- SLIs and SLOs
- Alert design
- Capacity planning
- Backups
- Incident response
- Postmortems

Core lesson

Reliable systems are measured systems. Logs tell you what happened, metrics show trends and traces connect distributed operations. Use all three where complexity justifies them.

Service-level indicators describe measurable behavior such as availability or latency. Objectives define targets. Alerts should represent conditions that require action, not every unusual event.

Capacity planning connects workload to infrastructure. Monitor CPU, memory, storage, database connections and request rates. Load testing should be performed before major launches when possible.

Backups are useful only if they can be restored. Test restoration and document recovery time and recovery point objectives.

During an incident, stabilize first. Stop the harmful change, reduce traffic or disable a failing feature if necessary. Then diagnose and recover. A blameless postmortem should identify system causes and preventive actions.

Practical lab

- Create a dashboard for one service.
- Define three SLIs and SLOs.
- Configure meaningful alerts.
- Perform a backup and restore test.
- Write a postmortem for a simulated outage.

Assessment

Assessment: Design an incident response process for a production API with elevated error rates.

Module project

Project: Reliability Lab — monitoring dashboard, alert policy, backup/restore test and incident runbook.

Module 6 — Advanced Infrastructure and Zero-Trust Thinking

Learning objectives

- Scaling
- Caching and queues
- Serverless patterns
- Infrastructure automation
- Zero-trust principles
- Supply-chain security
- Cost optimization

Core lesson

As systems grow, synchronous request chains can become bottlenecks. Queues allow work to be processed asynchronously. Caching reduces repeated computation and database load. Horizontal scaling adds instances rather than making one machine endlessly larger.

Serverless functions are useful for event-driven tasks but have trade-offs around startup time, runtime limits and observability. Choose based on workload.

Zero-trust thinking assumes that network location alone does not imply trust. Authenticate and authorize requests explicitly. Segment services and minimize privileges.

Supply-chain security covers source dependencies, build systems, container images and deployment credentials. Protect CI/CD because compromise there can become a production compromise.

Cloud cost optimization starts with visibility. Tag resources, measure utilization and remove unused capacity. Use appropriate storage classes and compute sizes. Reliability and security should not be sacrificed merely to reduce cost.

Practical lab

- Design an event-driven architecture with a queue.
- Add caching to a read-heavy service.
- Create an automated infrastructure deployment.
- Map trust boundaries in a microservice system.
- Produce a monthly cloud cost review.

Assessment

Assessment: Design a scalable architecture and explain where queues, caches and autoscaling are useful.

Module project

Project: Scalable Platform Lab — asynchronous processing, caching, autoscaling plan, secure identities and cost controls.

Module 7 — Capstone: Secure Production Platform

Learning objectives

- Architecture
- Infrastructure as code
- CI/CD
- Security controls
- Observability
- Disaster recovery
- Professional operations

Core lesson

The capstone is a production-style platform for a web or AI application. Start with requirements: expected traffic, availability target, data sensitivity, deployment frequency and recovery objectives.

Create infrastructure that can be recreated from code. Implement CI/CD with automated tests and controlled production deployment. Use least-privilege identities and a secret manager.

Add observability from day one. Build dashboards for request rate, latency, errors, resource use and important business events. Define alerts and an incident runbook.

Test failure intentionally in a safe environment: stop a service, simulate a dependency timeout or deploy a known-bad version. Verify that health checks, rollback and recovery work.

The final deliverable should include architecture diagrams, infrastructure code, security review, monitoring dashboard, backup plan, cost estimate and operational handbook. This demonstrates practical engineering far beyond a cloud certification alone.

Practical lab

- Define platform requirements.
- Create infrastructure as code.
- Build secure CI/CD.
- Add monitoring and recovery procedures.
- Run a controlled failure exercise.

Assessment

Assessment: Conduct a production readiness review and defend security, reliability, cost and scaling choices.

Module project

Capstone: Secure Cloud Platform — deploy a real application with infrastructure automation, CI/CD, monitoring, security hardening and disaster recovery documentation.

Instructor & Commercial Packaging Guide

- Turn each module into 3–6 video lessons plus one guided lab.
- Provide a downloadable starter repository and a separate completed reference implementation.
- Add quizzes after each lesson and a practical assessment after each module.
- Keep project requirements stable while allowing learners to choose their own domain.
- For commercial delivery, add your own branding, instructor examples, screenshots, exercises and community/support layer.
- Avoid promising jobs, income or certification unless you actually provide the relevant accreditation or service.

Suggested Course Completion Certificate Criteria

- Complete at least 5 of 7 module projects.
- Pass the capstone assessment.
- Submit a working portfolio artifact and technical documentation.
- Demonstrate the ability to explain design decisions and limitations.

Final Portfolio Checklist

- Public or private demo URL where appropriate.
- Architecture diagram.
- README with setup and usage.
- Test/evaluation evidence.
- Security and limitations section.
- Short case study describing the problem and measurable result.